CIVICHUB

AS-BUILT PRODUCT & SYSTEM DESIGN

VERSION 2.1.0

---

1. PRODUCT OVERVIEW

Document	As-built implementation reference	**Last Updated**	13 October 2026
Status	Reflects the current frontend and backend in this repository	**Project**	CivicHub municipal services demo

Purpose

CivicHub is a responsive municipal services web application themed around Tel Aviv-Yafo. It brings resident self-service, municipal staff workflows and field-guard tools into one role-based interface. This repository contains a React frontend and an Express API in `Backend/`. The API connects to MongoDB and optional Cloudinary and Novu services; the database and those external services must be configured separately.

Product Goals

- Give residents a clear view of their account, property, utility usage, invoices, fines and service requests.
- Give municipal staff a single workspace for resident reports, maintenance dispatch, citation disputes and a billing ledger.
- Support field guards with report handling, citation history and a parking-fine form.
- Keep the experience usable on desktop and mobile, with light and dark themes.

Scope and Trust Boundaries

The repository contains the browser application and API source, but not a MongoDB server or the hosted Cloudinary and Novu services. Billing/payment is record-keeping only: there is no payment gateway or card processing. Some portal content remains seeded demonstration data, so distinguish those examples from API-backed records. See section 5 for the current persistence boundaries.

2. IMPLEMENTED CAPABILITIES

Area	Current behavior	Data / persistence boundary
Sign-in and roles	Login screen, one-click demo access for Resident, Staff and Field Guard, role-aware route protection and sign-out.	Demo identities are client-side. Standard sign-in and session restoration call the local API.
Resident portal	Dashboard, property details, water/electricity usage chart, invoices and PDF receipts, bill/fine payment recording, service reports, fines and citation appeals.	Reports, fines, utility account data, bill/fine payment records and appeals use the API. Some property and dashboard presentation data is seeded in the frontend. Payment recording does not charge a payment method.
Municipal staff	Overview, searchable report inbox, report assignment or rejection, maintenance status board, citation-dispute decisions and billing ledger.	Reports, appeals/decisions, utility accounts and fines are loaded from or changed through the API. Maintenance tickets are derived from API-backed reports. Portal navigation and display state remain client-side.
Field Guard	Overview, report list and progress actions, issue history, and a form for a parking fine with plate validation and a selected violation type.	Report handling and fine creation use the API. Fine photos can be uploaded to the configured image service. Whether records constitute official municipal enforcement depends on the deployment and its operational authorization.
Shared experience	Responsive navigation, mobile drawer, theme toggle, toast messages and Novu in-app notification component.	Theme and page state are client-side. Novu requires frontend application/subscriber configuration and a backend secret key for backend notification workflows.

3. ARCHITECTURE & TECHNOLOGY

Implemented Application Stack

- **Application:** React 19, TypeScript 6, Vite 8 and React Router 7.
- **Client state:** React Context for user, theme and citation-related state; component state for portal tabs, forms and transient interactions.
- **UI and mapping:** Recharts for utility charts, Leaflet for the report-location map, lucide-react for icons, Sonner for toast feedback and Novu React for the notification inbox.
- **Documents:** jsPDF and jspdf-autotable for invoice receipt PDFs.
- **Styling and frontend tooling:** plain CSS (including shared mobile styles), TypeScript, Vite, ESLint and the Vite React plugin.
- **Backend:** Node.js with native ES modules, Express 5, Mongoose/MongoDB, JWT authentication, bcrypt password hashing, Multer and Cloudinary uploads, plus Novu API/workflow packages.

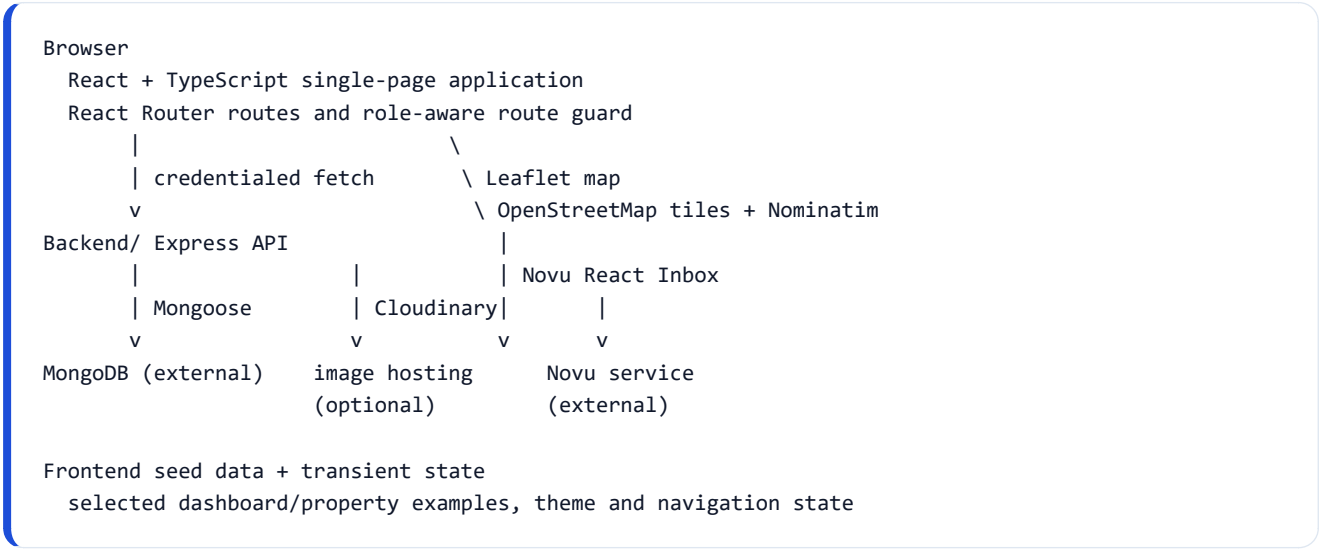
Dependency Inventory and Explicit Non-Dependencies

Area	Declared packages	What they do here
Frontend runtime	`react`, `react-dom`, `react-router-dom`, `recharts`, `leaflet`, `lucide-react`, `sonner`, `@novu/react`, `jspdf`, `jspdf-autotable`	UI/routing, charts, map, icons, toasts, notifications and receipt PDFs.
Backend runtime	`express`, `mongoose`, `jsonwebtoken`, `bcrypt`, `cookie-parser`, `cors`, `dotenv`, `multer`, `cloudinary`, `@novu/api`, `@novu/framework`	HTTP API, MongoDB models, cookie/JWT authentication, password hashing, CORS/ configuration, uploads and notifications.
Build and lint tooling	`vite`, `@vitejs/plugin-react`, `typescript`, `eslint`, `typescript-eslint`, `@eslint/js`, `eslint-plugin-react-hooks`, `eslint-plugin-react-refresh`, `globals`	Frontend development/build, type checking and linting; these are not application runtime services.
Type packages	`@types/react`, `@types/react-dom`, `@types/leaflet`, `@types/node`, `@types/express`	Development-time TypeScript declarations.

Not used / not declared: the app uses browser `fetch`, not Axios or another API-client package; state uses React Context rather than Redux or Zustand; styling is CSS rather than Tailwind or Bootstrap. `react-toastify` is present in the frontend dependency manifest but is not imported by the application; toast UI is implemented with `sonner`. No payment processor is integrated.

External services: Leaflet map tiles use OpenStreetMap's tile service and address lookup uses Nominatim. Cloudinary handles uploaded images when configured. Novu provides notification infrastructure when configured. These services are not bundled in this repository.

Runtime Shape



The API source, route handlers, Mongoose models and service configuration are in `Backend/`. MongoDB, Cloudinary and Novu are external dependencies that require deployment credentials/configuration. The frontend currently hard-codes the API origin to `http://localhost:4000`; credentialed CORS is configured for local Vite origins only. This document does not assert production hosting, service-level targets or a production security review.

The Mongoose model layer defines users, resident profiles, reports, fines, appeals, utility accounts and vehicles. Payment and appeal workflows use MongoDB transactions, so the configured MongoDB deployment must support transactions.

Frontend Organization

- `src/pages` contains route-level screens.
- `src/components` contains login, resident, staff, field-guard and shared UI.
- `src/layouts` owns staff and field-guard navigation and shared portal state.
- `src/contexts` provides user, theme and citation state.
- `src/data` contains typed example data and frontend view types.
- `src/helpers` contains formatting, PDF and field-guard helpers; `src/styles` contains page and shared CSS.
- `Backend/config`, `Backend/controllers`, `Backend/middleware`, `Backend/models`, `Backend/routes`, `Backend/services`, `Backend/utils` and `Backend/workflows` contain API configuration, handlers, persistence models and integrations.

4. ROUTES & ROLE WORKSPACES

Path	Role	Screen and navigation
`/login`	Public	Login form and demo account shortcuts.
`/resident`	`RESIDENT`	Single route with Dashboard, Billing, My Services and Properties tabs.
`/staff`	`STAFF`	Staff shell with Requests, Disputes, Billing and Maintenance tabs.
`/field-guard`	`FIELD_GUARD`	Field-guard shell with Dashboard, Issue Citation, Reports and History tabs. Other nested field-guard paths redirect to this route.
`/` and unmatched paths	Any	The root redirects to login; unknown paths show the not-found screen.

Protected routes check the current user role in the frontend and redirect unauthenticated users to login or users with the wrong role to their own portal. This improves navigation behavior; server-side authorization remains the responsibility of the API.

5. DATA & INTERACTION STATE

Data Sources

- **Seeded examples:** Some resident/property presentation values and portal examples originate in `src/data/residentPortal.ts`; field-guard display data also has examples in `src/data/fieldGuardData.ts`. These should not be confused with records stored in MongoDB.
- **API-backed records:** Authentication, reports, fines, appeals and their decisions, utility account data, and payment-record updates use the local API. The API persists records through Mongoose models in MongoDB.
- **Transient frontend state:** Portal navigation, theme, open forms and visual interaction state are held in React state/context. Persistent API changes are loaded from the API rather than being only browser-memory changes.

Report Lifecycle Used by the UI

```
NEW -> DISPATCHED -> IN PROGRESS -> RESOLVED
  \                        \----> REJECTED
   \--------------------------> REJECTED

Staff can assign a report to Maintenance or a Field Guard.
Field Guard can update an eligible assigned report's progress,
resolve it, or reject it with a reason.
```

The displayed labels map backend report statuses to friendlier portal terms (for example, `NEW` to "Pending" for staff and "Reported" for residents). The API remains the source of truth for API-backed reports.

Important Demo Boundaries

- Payment completion calls `PUT /utilities/pay` to mark selected utility months and/or fines as paid in the API. It does not connect to a bank, card network or payment provider; do not enter real payment-card details.
- Resident appeals and staff decisions use `POST /appeals` and `PUT /appeals/:id/decision`; fine and report submissions also call the API. Uploaded images depend on configured Cloudinary credentials.
- Some dashboard/property content is seeded demonstration data and may not represent the signed-in account's authoritative municipal record.
- API requests target the hard-coded local origin `http://localhost:4000`. A non-local deployment needs configurable origins, production CORS settings and external-service configuration.

6. FRONTEND API CONTRACT

The following are the API calls currently made by this frontend. They use `http://localhost:4000` directly and include browser credentials. The API implementation is in `Backend/`; this table covers frontend-used routes, not every route exposed by the API.

Method and path	Frontend use
`POST /auth/register`	Create a resident account.
`GET /auth/me`	Restore the signed-in user on app startup.
`POST /auth/login`	Submit email and password; response provides the user and role.
`POST /auth/logout`	End a non-demo session.
`GET /appeals`	Load resident appeals or the staff dispute queue.
`POST /appeals`	Submit a citation appeal, optionally with a photo.
`PUT /appeals/:id/decision`	Save a staff accept/reject decision.
`GET /fines`	Load the current resident's or field guard's fines.
`POST /fines`	Create a field-guard fine, optionally with a photo.
`GET /fines/staff`	Load fines for the staff billing ledger.
`GET /utilities` and `GET /utilities/me`	Load the staff utility ledger or current resident's utility account.
`PUT /utilities/pay`	Record selected utility-month and/or fine payments; no external payment processing occurs.
`GET /reports`	Load reports for resident, staff and field-guard views.
`POST /reports`	Submit a resident service report as multipart form data, optionally including an image (JPEG, PNG or WebP, maximum 5 MB).
`GET /reports/field-guards`	Load field-guard choices for staff assignment.
`PUT /reports/:id/assign`	Assign a report to the maintenance team or a selected field guard.
`PUT /reports/:id/reject`	Reject a pending staff report with a reason.
`PUT /reports/:id/field-guard-status`	Update an assigned report to in progress, resolved or rejected.
`POST /reports/:id/maintenance-progress`	Add a maintenance progress note.
`PUT /reports/:id/maintenance-status`	Update maintenance progress from the staff dispatch board.

Authentication and role authorization are implemented in the API as well as reflected in frontend route behavior. For a non-local deployment, the API origin and allowed CORS origins must be configurable, and

the server's database and external-service settings must be supplied securely.

7. UX, RESPONSIVE DESIGN & EXTERNAL LIBRARIES

- **Responsive portals:** Dedicated mobile drawer and compact layouts for resident, staff and field-guard screens; table-like content adapts to narrow viewports.
- **Theme:** Shared light/dark theme context and toggle, including chart colors that respond to the active theme.
- **Maps and address lookup:** Leaflet renders the resident report location picker, OpenStreetMap supplies map tiles, and Nominatim reverse geocodes selected coordinates.
- **Data visualization:** Recharts displays utility usage on the resident dashboard; it is not a staff revenue or enforcement analytics dashboard.
- **Receipts:** jsPDF and jspdf-autotable generate downloadable invoice receipts from frontend invoice data.
- **Notifications:** The shared notification component uses Novu React. Configure `VITE_NOVU_APPLICATION_IDENTIFIER` and subscriber ID/hash; backend notification workflows also require `NOVU_SECRET_KEY`.
- **Feedback and semantics:** Forms expose validation and loading/error/success feedback, with labels and status/alert roles used on key interactions. No formal WCAG conformance claim has been tested or is made here.

8. LOCAL SETUP & VERIFICATION

Run the Frontend

Use Node.js 20 or newer. From the repository root:

```
npm install
npm run dev
```

Vite prints the local URL. Demo portal shortcuts can be used to explore the UI without signing in through the API. API-backed workflows require the backend below to be available at `http://localhost:4000`.

Run the Backend

In a second terminal, install the backend dependencies in `Backend/`, configure its `.env` with `PORT=4000`, `MONGO_URI` and `JWT_SECRET`, then start the API:

```
cd Backend
npm install
npm run dev
```

Cloudinary credentials are required for image storage; Novu requires `NOVU_SECRET_KEY` when backend notification code is initialized. MongoDB must support transactions (for example, Atlas or a local replica set). The backend package's `dev` script calls `nodemon`, but `nodemon` is not currently declared in `Backend/package.json`; declare/install it before relying on that script.

Repository Scripts

Command	Purpose
`npm run dev`	Start the Vite development server.
`npm run build`	Run TypeScript project builds and create the production frontend bundle.
`npm run lint`	Run ESLint.
`npm run preview`	Serve the built frontend locally for preview.

Known Follow-up Work

- Move the API base URL and allowed CORS origins into environment configuration before deploying outside local development.
- Integrate an authorized payment provider before representing payment records as completed financial transactions.
- Replace remaining seeded dashboard and property examples with authoritative API-backed data where production behavior is intended.
- Review demo copy and data handling so users can clearly distinguish browser-only examples from reports sent to the configured API.
- Run accessibility, security and end-to-end workflow reviews before production use; this document does not assert those reviews have passed.

Project Attribution

The project README credits Helayel Hamayel and Mohamad Amer.